

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG  
KHOA CÔNG NGHỆ THÔNG TIN 1



**BÁO CÁO HÀNG TUẦN**

**Xây dựng Chatbot hỏi đáp về lịch sử Việt Nam sử dụng  
GraphRAG**

**Lớp:** E22HTTT

**Tên:** Nguyễn Đức Anh

**ID:** B22DCDT017

**Người hướng dẫn:** Kim Ngọc Bách

Hà Nội – 2026

## Weekly Report 13

Trong tuần vừa qua em bắt đầu viết báo cáo cuối kì

# CHƯƠNG 1. GIỚI THIỆU CHUNG

## 1.1. Đặt vấn đề

Lịch sử dân tộc là kho tàng tri thức vô cùng quý giá, phản ánh quá trình đấu tranh dựng nước và giữ nước hào hùng của nhân dân Việt Nam. Tuy nhiên, việc học và ôn tập lịch sử theo phương pháp truyền thống — đọc sách giáo khoa, ghi chép thụ động — thường gây ra cảm giác nhàm chán, khó ghi nhớ và thiếu tính tương tác cho người học.

Trong bối cảnh Trí tuệ nhân tạo (AI) và Xử lý ngôn ngữ tự nhiên (NLP) đang phát triển vượt bậc, việc ứng dụng các công nghệ tiên tiến như Mô hình ngôn ngữ lớn (LLM), Đồ thị tri thức (Knowledge Graph) và Tìm kiếm ngữ nghĩa (Semantic Search) vào giáo dục là hướng đi đầy tiềm năng và cấp thiết.

Đề án này ra đời từ chính nhu cầu đó: xây dựng một hệ thống chatbot hỏi đáp lịch sử thông minh, không chỉ truy xuất thông tin chính xác mà còn hiểu được các mối quan hệ nhân quả, bối cảnh lịch sử và logic sự kiện — điều mà các hệ thống tìm kiếm thông thường không thể làm được.

## 1.2. Mục tiêu đề án

Đề án hướng tới các mục tiêu cụ thể sau:

- Xây dựng pipeline thu thập, xử lý và chuẩn hóa dữ liệu lịch sử Việt Nam giai đoạn 1954–1975 từ nhiều nguồn khác nhau.
- Triển khai hệ thống Hybrid RAG kết hợp Đồ thị tri thức Neo4j (GraphRAG) và Tìm kiếm vector ngữ nghĩa (VectorRAG).

- Tích hợp mô hình AI (GPT-4o-mini) để tổng hợp và trả lời câu hỏi lịch sử theo phong cách Giáo sư chuyên nghiệp.
- Phát triển giao diện web tương tác thân thiện bằng Streamlit, hỗ trợ lịch sử hội thoại và chức năng kiểm chứng dữ liệu nguồn.

### 1.3. Phạm vi nghiên cứu

Đề án tập trung vào:

- Nội dung lịch sử: Cuộc kháng chiến chống Mỹ, cứu nước (1954–1975), bao gồm các sự kiện, nhân vật, chiến lược, địa danh và kết quả lịch sử trong giai đoạn này.
- Nguồn dữ liệu: Sáu tài liệu văn bản tiếng Việt từ Wikipedia, tài liệu Đại học Kinh tế TP.HCM, Báo Hà Tĩnh, Thư viện Pháp luật và tài liệu học thuật được công bố công khai.
- Công nghệ: Neo4j Aura (cloud), OpenAI API, Google Gemini API, LangChain, Streamlit.

### 1.4. Ý nghĩa của đề án

Về mặt học thuật, đề án đóng góp một kiến trúc Hybrid RAG có thể áp dụng cho nhiều lĩnh vực tri thức chuyên biệt tương tự. Về mặt ứng dụng thực tiễn, hệ thống có thể được tích hợp vào nền tảng học tập trực tuyến (e-learning) để hỗ trợ học sinh ôn thi lịch sử một cách tương tác và hiệu quả.

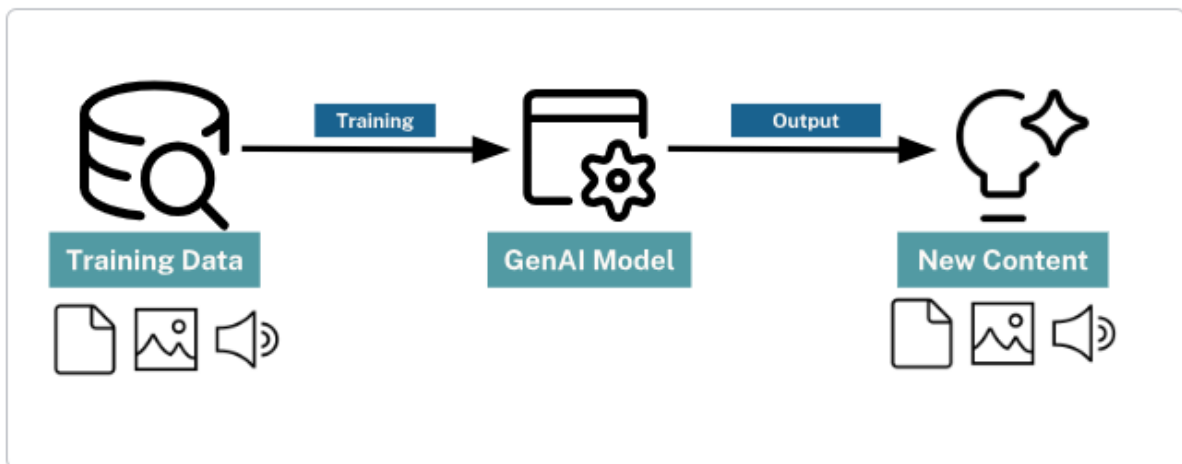
## Chương 2: Cơ sở lý thuyết và công nghệ sử dụng

### 2.1. Trí tuệ nhân tạo sinh (Generative AI)

Trí tuệ nhân tạo sinh, hay Generative AI, là nhóm hệ thống trí tuệ nhân tạo được thiết kế để tạo ra nội dung mới có đặc điểm giống dữ liệu do con người tạo ra. Nội dung được sinh ra có thể là văn bản, hình ảnh, âm thanh, mã nguồn hoặc nhiều dạng dữ liệu khác. Trong bối cảnh project này, Generative AI được sử dụng chủ yếu để hiểu câu hỏi bằng tiếng Việt, hỗ trợ truy vấn dữ liệu và tạo câu trả lời tự nhiên cho người học lịch sử.

Các mô hình Generative AI được huấn luyện trên tập dữ liệu rất lớn. Trong quá trình huấn luyện, mô hình học các mẫu, quy luật ngôn ngữ, mối liên hệ ngữ nghĩa và cách diễn đạt thường gặp. Khi nhận đầu vào mới, mô hình dự đoán đầu ra có khả năng phù hợp nhất dựa trên những mẫu đã học. Quy trình khái quát có thể hiểu là: dữ liệu huấn luyện được đưa vào mô hình, mô hình học biểu diễn tri thức từ dữ liệu, sau đó sinh nội dung mới khi người dùng đặt câu hỏi hoặc yêu cầu.

Trong hệ thống chatbot lịch sử Việt Nam, Generative AI không được dùng như một nguồn kiến thức độc lập tuyệt đối. Thay vào đó, mô hình được đặt trong kiến trúc RAG và GraphRAG để câu trả lời được neo vào dữ liệu lịch sử đã thu thập, giúp giảm tình trạng trả lời chung chung hoặc bịa đặt thông tin.



### 2.2. Mô hình ngôn ngữ lớn (Large Language Models - LLMs)

Mô hình ngôn ngữ lớn, gọi tắt là LLM, là một loại mô hình Generative AI chuyên xử lý, hiểu và sinh văn bản giống ngôn ngữ tự nhiên của con người. LLM có thể thực hiện

nhiều tác vụ như trả lời câu hỏi, tóm tắt tài liệu, phân tích văn bản, viết lại nội dung, sinh mã nguồn và hỗ trợ hội thoại.

LLM khác với nhiều mô hình AI truyền thống ở chỗ nó có khả năng xử lý ngôn ngữ linh hoạt trên nhiều miền tri thức khác nhau. Thay vì chỉ dự đoán một nhãn cố định, LLM có thể tạo ra một đoạn trả lời hoàn chỉnh. Tuy nhiên, phản hồi của LLM về bản chất vẫn là kết quả dự đoán xác suất dựa trên dữ liệu huấn luyện và ngữ cảnh được cung cấp trong prompt. Vì vậy, chất lượng câu trả lời phụ thuộc nhiều vào dữ liệu đầu vào, prompt và cơ chế kiểm soát ngữ cảnh.

Trong project, LLM được sử dụng ở hai vai trò chính. Thứ nhất, mô hình hỗ trợ chuyển câu hỏi tự nhiên của người dùng thành truy vấn Cypher để lấy dữ liệu từ Neo4j Knowledge Graph. Thứ hai, mô hình nhận dữ liệu truy hồi từ đồ thị và vector search để tổng hợp câu trả lời cuối cùng bằng tiếng Việt.

### 2.3. Hạn chế của Generative AI và LLM

Mặc dù LLM có khả năng tạo văn bản rất tốt, mô hình vẫn tồn tại nhiều hạn chế quan trọng. Các hạn chế này là lý do cần kết hợp LLM với RAG, Knowledge Graph và cơ sở dữ liệu ngoài trong hệ thống hỏi đáp lịch sử.

- Hạn chế về dữ liệu: dữ liệu huấn luyện thường đến từ internet, sách và nguồn công khai. Những nguồn này có thể thiếu cập nhật, không đầy đủ hoặc không chứa dữ liệu riêng của project. Nếu câu hỏi yêu cầu thông tin không có trong dữ liệu huấn luyện, mô hình có thể trả lời thiếu chính xác.
- Hạn chế về thời điểm tri thức: quá trình huấn luyện diễn ra tại một thời điểm nhất định, nên tri thức của mô hình mang tính tĩnh. Mô hình không tự động biết những dữ liệu mới hoặc dữ liệu nội bộ nếu chúng không được đưa vào prompt hoặc hệ thống truy hồi.
- Hiện tượng hallucination: LLM có xu hướng tạo câu trả lời trôi chảy và có vẻ hợp lý, nhưng đôi khi nội dung có thể không có căn cứ. Đây là vấn đề đặc biệt nguy hiểm trong môn Lịch sử vì chỉ cần sai tên nhân vật, mốc thời gian hoặc quan hệ sự kiện là câu trả lời mất giá trị học tập.
- Tham số temperature: temperature cao giúp câu trả lời đa dạng và sáng tạo hơn, nhưng có thể làm giảm tính ổn định. Với chatbot học lịch sử, cần ưu tiên độ chính xác nên temperature thường được đặt thấp để câu trả lời nhất quán.
- Tính minh bạch: mô hình GenAI thường được xem là hộp đen, khó giải thích đầy đủ vì sao mô hình chọn một câu trả lời cụ thể. Do đó, hệ thống cần hiển thị dữ liệu truy hồi hoặc đường dẫn quan hệ để người dùng kiểm chứng.

Từ các hạn chế trên, project thiết kế final prompt yêu cầu mô hình chỉ trả lời dựa trên dữ liệu truy xuất được. Nếu dữ liệu không có thông tin, chatbot phải thừa nhận chưa có dữ liệu thay vì tự suy đoán.

#### **2.4. Vai trò của ngữ cảnh trong việc cải thiện câu trả lời**

Một cách quan trọng để nâng cao độ chính xác của LLM là cung cấp ngữ cảnh phù hợp trong prompt. Ngữ cảnh có thể là đoạn tài liệu, dữ liệu thống kê, kết quả truy vấn cơ sở dữ liệu, quan hệ giữa các thực thể hoặc các ràng buộc trả lời. Khi có ngữ cảnh, mô hình không phải dựa hoàn toàn vào kiến thức đã học mà có thể căn cứ trực tiếp vào dữ liệu được cung cấp.

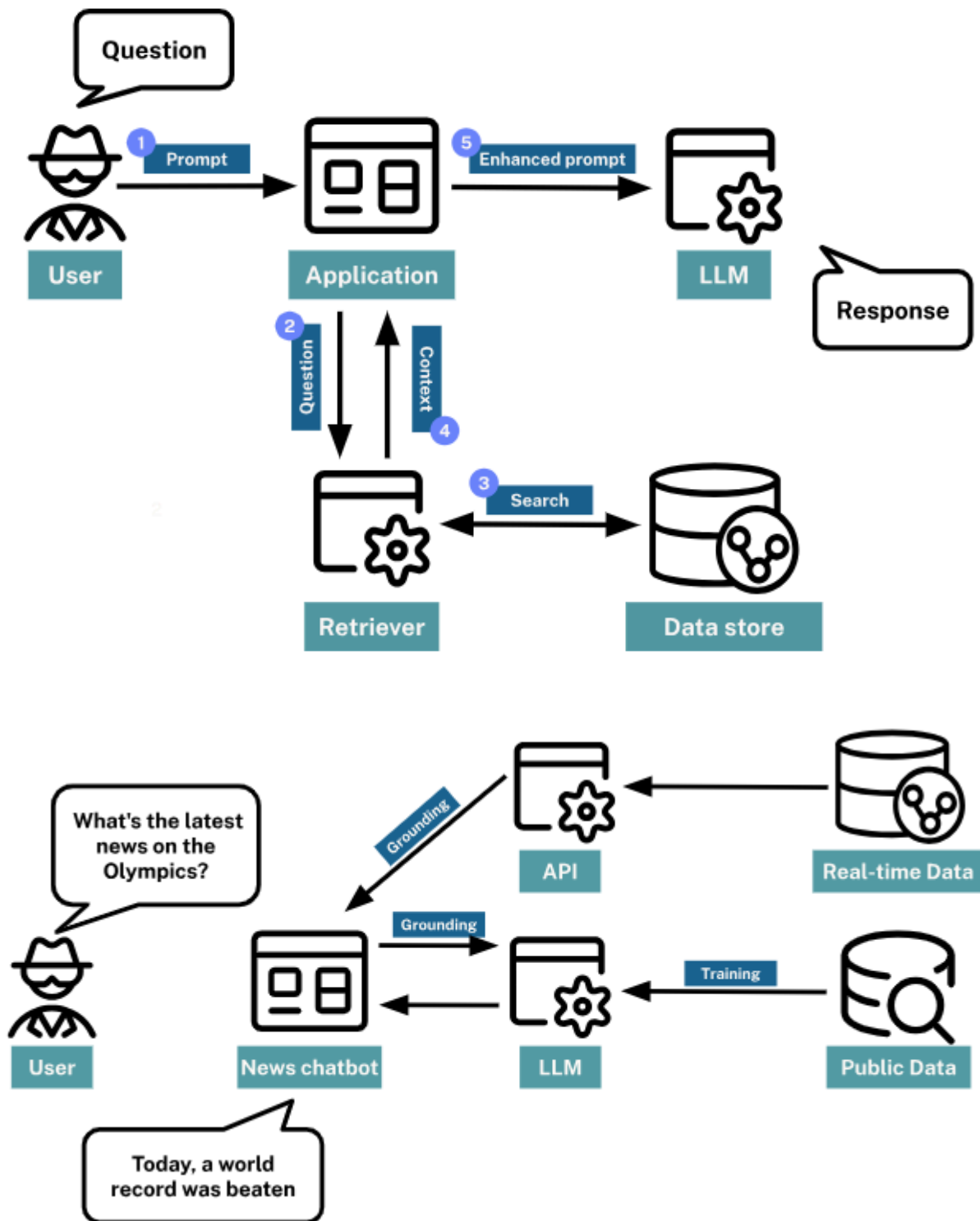
Cung cấp ngữ cảnh giúp giảm hallucination vì câu trả lời được neo vào các sự kiện, chi tiết và thông tin cụ thể. Với project chatbot lịch sử Việt Nam, ngữ cảnh bao gồm hai nhóm: văn bản lịch sử được truy hồi bằng vector search và dữ liệu quan hệ được truy hồi từ Neo4j Knowledge Graph. Hai nguồn này bổ sung cho nhau: văn bản cung cấp diễn giải chi tiết, còn đồ thị cung cấp cấu trúc thực thể - quan hệ rõ ràng.

#### **2.5. Retrieval-Augmented Generation (RAG)**

Retrieval-Augmented Generation, viết tắt là RAG, là phương pháp kết hợp truy hồi thông tin với sinh ngôn ngữ. Thay vì để LLM trả lời chỉ dựa trên tri thức đã học, hệ thống trước hết tìm kiếm thông tin liên quan từ nguồn dữ liệu bên ngoài, sau đó đưa thông tin này vào prompt để LLM tạo câu trả lời. Cách làm này giúp câu trả lời chính xác hơn, cập nhật hơn và phù hợp hơn với dữ liệu riêng của ứng dụng.

Quy trình RAG thường gồm ba bước chính:

1. Hiểu câu hỏi của người dùng: hệ thống phân tích câu hỏi để xác định người dùng đang cần thông tin gì.
2. Truy hồi thông tin: bộ truy hồi tìm kiếm dữ liệu liên quan trong tài liệu, cơ sở dữ liệu, API hoặc knowledge graph.
3. Sinh câu trả lời: thông tin truy hồi được đưa vào prompt, sau đó LLM sử dụng ngữ cảnh này để tạo câu trả lời phù hợp.



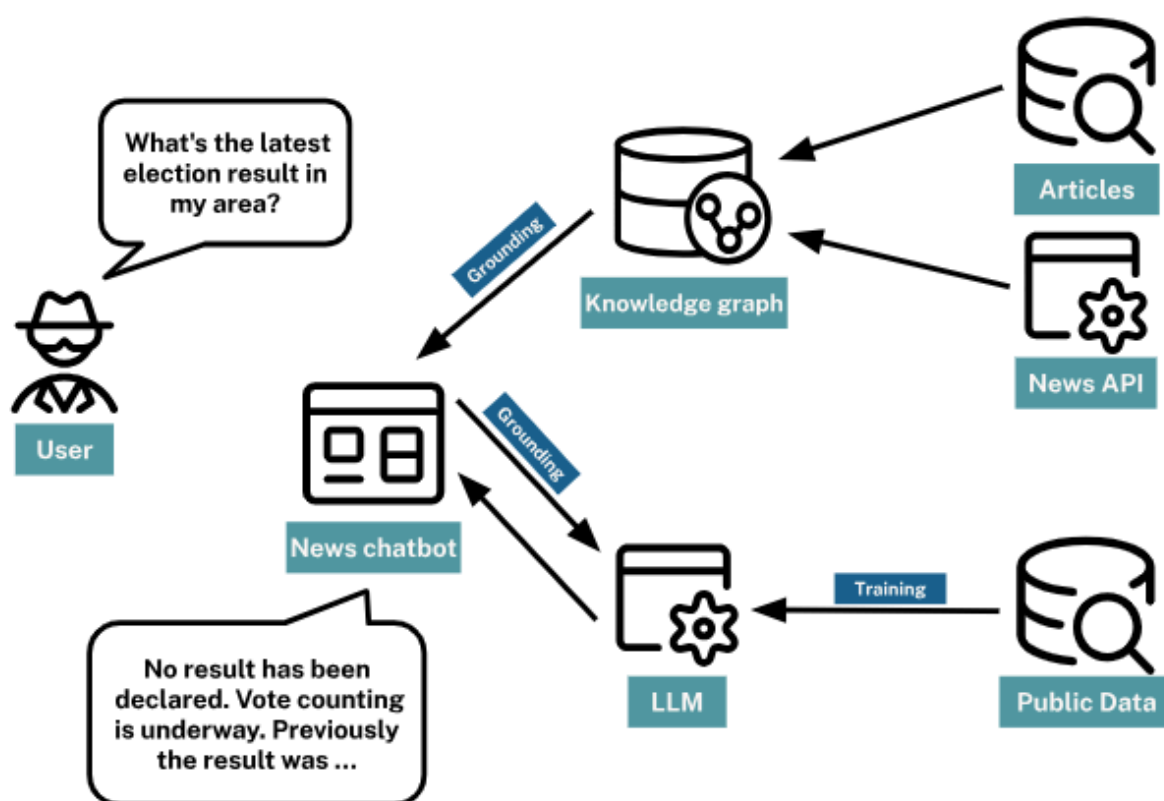
Trong project, RAG được triển khai bằng cách tách các tài liệu lịch sử thành chunk, sinh embedding cho từng chunk và lưu vào Neo4j dưới dạng node Chunk. Khi người dùng đặt câu hỏi, hệ thống tạo embedding cho câu hỏi và tìm các chunk có độ tương đồng cao nhất. Các chunk này trở thành ngữ cảnh văn bản cho LLM.

## 2.6. Bộ truy hồi và nguồn dữ liệu trong RAG

Retriever, hay bộ truy hồi, là thành phần chịu trách nhiệm tìm và lấy thông tin liên quan từ nguồn dữ liệu bên ngoài dựa trên câu hỏi của người dùng. Bộ truy hồi thường nhận đầu vào dạng không cấu trúc, ví dụ một câu hỏi tự nhiên, sau đó tìm dữ liệu có khả năng cung cấp ngữ cảnh hoặc câu trả lời.

Các nguồn dữ liệu phổ biến trong hệ thống RAG gồm:

- Tài liệu văn bản: bài viết, báo cáo, sách, giáo trình, file TXT hoặc PDF có thể được tách đoạn và tìm kiếm.
- API: dịch vụ bên ngoài cung cấp dữ liệu thời gian thực hoặc dữ liệu chuyên biệt theo truy vấn.
- Knowledge Graph: biểu diễn thông tin dưới dạng thực thể và quan hệ, phù hợp để trả lời các câu hỏi về liên kết giữa sự kiện, nhân vật, tổ chức và địa danh.
- Cơ sở dữ liệu tài liệu hoặc cơ sở dữ liệu vector: nơi lưu các đoạn văn bản đã được embedding để phục vụ tìm kiếm ngữ nghĩa.



Đối với chatbot lịch sử Việt Nam, nguồn dữ liệu chính là các tài liệu tiếng Việt về giai đoạn kháng chiến chống Mỹ và lịch sử Việt Nam 1954-1975. Dữ liệu này được xử lý

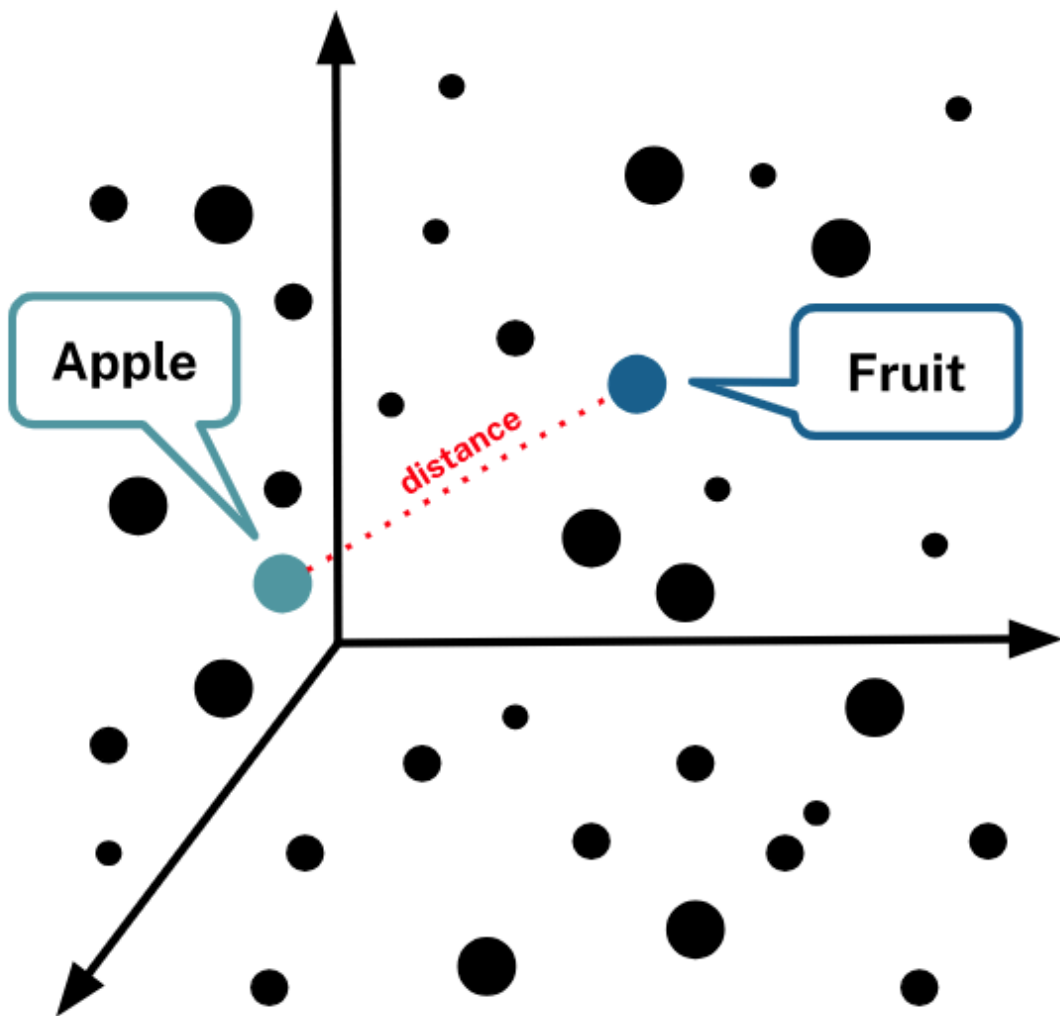
theo hai hướng song song: một hướng tạo vector cho các chunk văn bản, một hướng trích xuất node và edge để tạo knowledge graph.

## **2.7. Tìm kiếm ngữ nghĩa, vector và embedding**

Tìm kiếm ngữ nghĩa, hay semantic search, là phương pháp tìm kiếm dựa trên ý định và ý nghĩa ngữ cảnh của câu hỏi thay vì chỉ so khớp từ khóa chính xác. Tìm kiếm từ khóa truyền thống thường phụ thuộc vào việc từ trong truy vấn có xuất hiện trong tài liệu hay không. Trong khi đó, semantic search có thể tìm được các đoạn liên quan dù cách diễn đạt khác nhau.

Để thực hiện semantic search, dữ liệu được biểu diễn thành vector. Vector là một danh sách các số, ví dụ [1, 2, 3] là một vector ba chiều. Trong xử lý ngôn ngữ tự nhiên, vector biểu diễn văn bản thường được gọi là embedding. Embedding là bản dịch số của văn bản, giúp máy tính so sánh hai đoạn văn bằng khoảng cách hoặc góc giữa các vector.

Mỗi chiều trong embedding có thể thể hiện một khía cạnh ngữ nghĩa nào đó của từ, câu hoặc đoạn văn. Khi nhiều chiều kết hợp với nhau, vector biểu diễn được ý nghĩa tổng thể của dữ liệu. Các văn bản có nội dung gần nhau sẽ có vector gần nhau; các văn bản không liên quan sẽ nằm xa nhau trong không gian vector.



Trong thực tế, embedding thường được tạo cho cả câu hoặc đoạn văn thay vì từng từ riêng lẻ, vì nghĩa của từ có thể thay đổi theo ngữ cảnh. Ví dụ trong tiếng Anh, từ “bank” có nghĩa khác nhau trong “river bank” và “savings bank”. Tương tự, trong tiếng Việt, một từ như “chiến dịch” có thể mang ý nghĩa khác nhau tùy giai đoạn, địa danh và lực lượng liên quan.

Trong project, dùng mô hình text-embedding-3-small để chuyển mỗi chunk lịch sử thành vector 1536 chiều. Các vector này được lưu trong Neo4j và được truy vấn bằng độ tương đồng cosine thông qua vector index chunk\_vector.

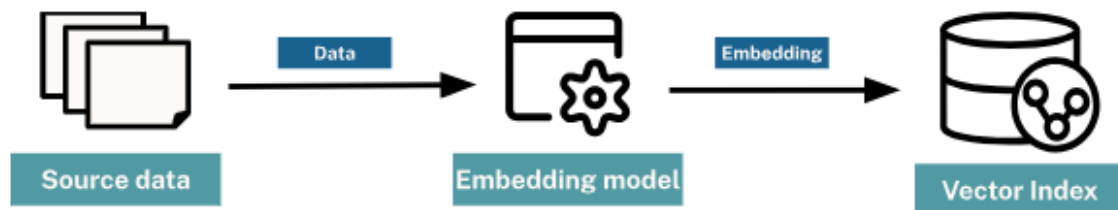
## 2.8. Vector Index trong Neo4j

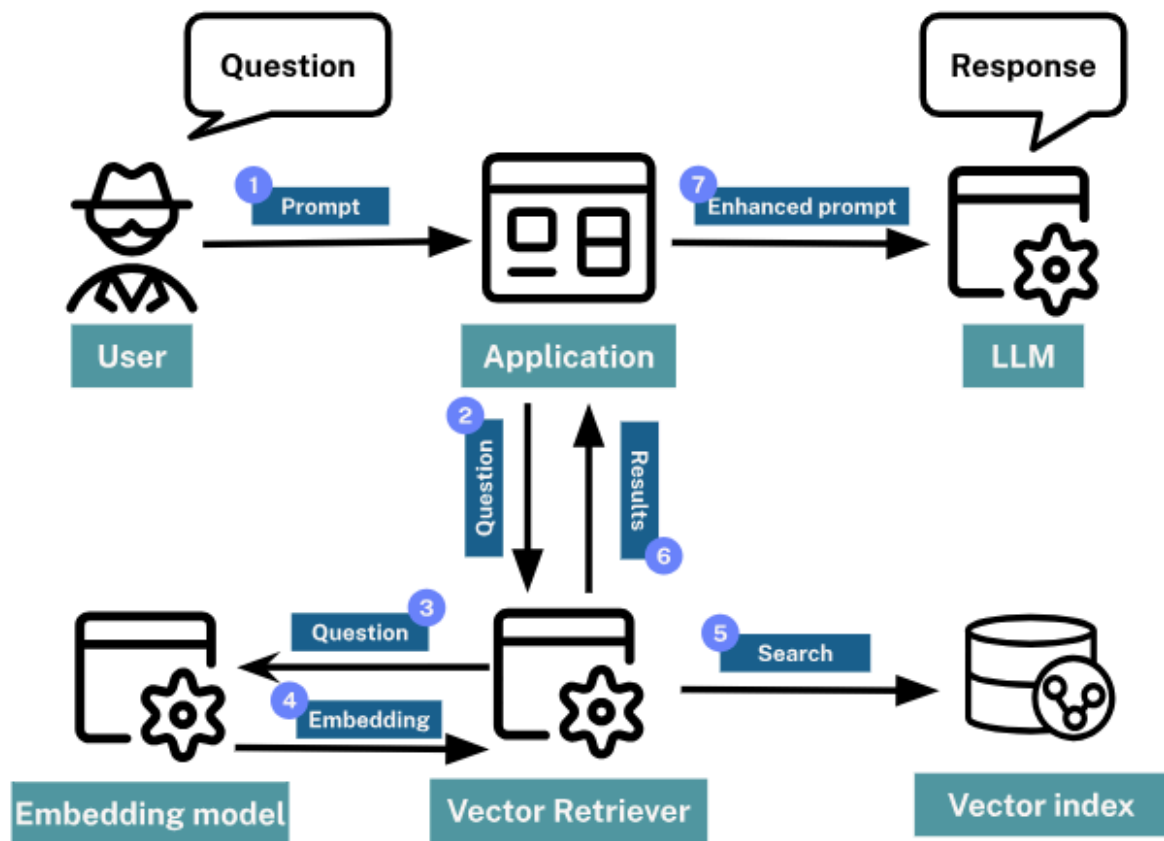
Vector index là cấu trúc chỉ mục giúp lưu trữ và tìm kiếm nhanh các vector embedding. Khi người dùng gửi câu hỏi, hệ thống tạo embedding cho câu hỏi, so sánh

vector câu hỏi với vector của các chunk đã lưu, chấm điểm độ tương đồng và trả về các chunk liên quan nhất để đưa vào prompt.

Vector search hoạt động tốt trong các trường hợp câu hỏi mang tính ngữ cảnh, câu hỏi mơ hồ, câu hỏi mở hoặc câu hỏi có nhiều khái niệm cần kết nối. Đây là kiểu câu hỏi phổ biến trong học lịch sử, ví dụ: “Vì sao Mỹ can thiệp vào Việt Nam?”, “Âm mưu của Mỹ ở Đông Dương là gì?”, hoặc “Miền Bắc có vai trò gì trong kháng chiến chống Mỹ?”.

Tuy nhiên, vector search không phải lúc nào cũng phù hợp. Nó kém hiệu quả hơn với câu hỏi yêu cầu khớp chính xác tuyệt đối, truy vấn số học, truy vấn logic kiểu đúng/sai hoặc câu hỏi quá mơ hồ nhưng thiếu ngữ cảnh. Vì vậy, project không chỉ dùng vector search mà còn kết hợp với truy vấn đồ thị bằng Cypher để tăng độ chính xác cho các câu hỏi về thực thể và quan hệ.





| Vector Search phù hợp với                  | Vector Search kém phù hợp với                       |
|--------------------------------------------|-----------------------------------------------------|
| Câu hỏi theo ngữ cảnh hoặc ý nghĩa         | Câu hỏi cần khớp chính xác từng ký tự               |
| Câu hỏi mơ hồ, diễn đạt gần nghĩa          | Truy vấn số liệu hoặc so sánh logic phức tạp        |
| Câu hỏi mở cần tìm đoạn tài liệu liên quan | Câu hỏi thiếu ngữ cảnh nghiêm trọng                 |
| Câu hỏi gồm nhiều khái niệm liên quan      | Tri thức chuyên biệt chưa có trong dữ liệu đã nhúng |

## 2.9. GraphRAG

GraphRAG là kỹ thuật mở rộng RAG bằng cách kết hợp tìm kiếm vector với dữ liệu đồ thị. Nếu RAG truyền thống chủ yếu đưa các đoạn văn bản liên quan vào prompt, GraphRAG bổ sung thêm các thực thể, quan hệ và đường liên kết trong knowledge graph. Điều này giúp hệ thống có ngữ cảnh phong phú hơn và có khả năng lý giải tốt hơn.

Các lợi ích chính của GraphRAG gồm:

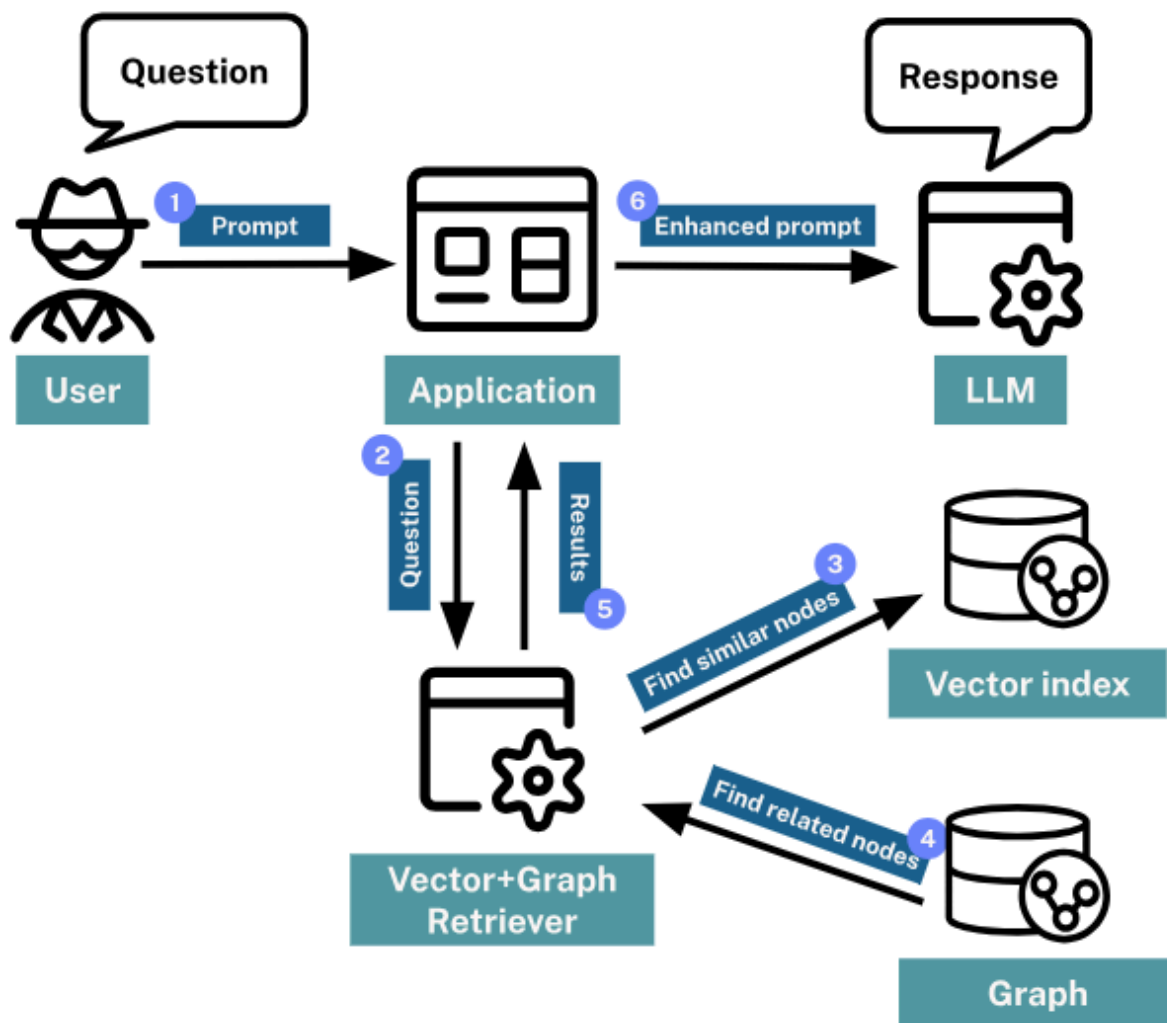
- Ngữ cảnh giàu hơn: đồ thị lưu được quan hệ giữa các thực thể, từ đó hệ thống có thể lấy thông tin không chỉ từ một đoạn văn bản mà còn từ các node và edge liên quan.
- Cải thiện độ chính xác: kết hợp độ tương đồng vector với truy vấn đồ thị giúp kết quả truy hồi chính xác và có bối cảnh hơn.
- Tăng khả năng giải thích: đồ thị cho thấy đường đi và quan hệ giữa các thực thể, giúp người dùng hiểu vì sao một kết quả được truy hồi.
- Hỗ trợ truy vấn linh hoạt: GraphRAG có thể kết hợp full-text search, vector search và text-to-Cypher search.
- Tăng khả năng suy luận: quan hệ trong đồ thị hỗ trợ phát hiện tri thức, gợi ý liên kết và trả lời các câu hỏi cần nối nhiều thông tin.

Trong project chatbot lịch sử Việt Nam, GraphRAG đặc biệt phù hợp vì dữ liệu lịch sử vốn có cấu trúc quan hệ dày đặc: nhân vật tham gia sự kiện, tổ chức lãnh đạo lực lượng, địa danh là nơi diễn ra chiến dịch, hiệp định tác động đến cục diện đất nước, hoặc quốc gia can thiệp vào một khu vực. Việc lưu trữ các thực thể và quan hệ này trong Neo4j giúp chatbot trả lời tốt hơn các câu hỏi dạng “ai liên quan đến ai”, “sự kiện nào dẫn đến sự kiện nào”, “tổ chức nào có âm mưu gì” hoặc “địa danh nào gắn với chiến dịch nào”.

## 2.10. Tìm kiếm vector kết hợp duyệt đồ thị

Một cách triển khai phổ biến của GraphRAG là kết hợp vector search và graph traversal. Quy trình mức cao gồm:

1. Người dùng gửi câu hỏi vào hệ thống.
2. Hệ thống dùng vector search để tìm các node hoặc chunk có nội dung gần với câu hỏi.
3. Từ các kết quả gần nhất, hệ thống duyệt đồ thị để tìm các thực thể, quan hệ hoặc node liên quan.
4. Dữ liệu văn bản, thực thể và quan hệ được đưa vào prompt làm ngữ cảnh cho LLM.
5. Hệ thống có thể chấm điểm hoặc lọc lại dữ liệu liên quan trước khi sinh câu trả lời.



Project hiện tại triển khai theo hướng hybrid: ứng dụng vừa gọi vector search để lấy chunk, vừa dùng Cypher và truy vấn quan hệ trong Neo4j. Kết quả của hai luồng được hợp nhất trong final prompt. Cách thiết kế này giúp câu trả lời vừa có phần diễn giải theo tài liệu gốc, vừa có dữ liệu quan hệ rõ ràng từ knowledge graph.

## 2.11. Full Text Search

Full text search là kỹ thuật tìm kiếm toàn văn dựa trên từ khóa hoặc cụm từ xuất hiện trong văn bản. Khác với vector search, full text search mạnh trong trường hợp người dùng cần tìm chính xác tên riêng, thuật ngữ hoặc cụm từ cụ thể. Ví dụ, nếu người dùng hỏi trực tiếp về “Ngô Đình Diệm”, “Hiệp định Giơ-ne-vơ 1954” hoặc “Chiến thắng Điện Biên Phủ”, full text search có thể nhanh chóng định vị các node hoặc đoạn văn bản chứa tên đó.

Full text search có thể được dùng thay thế hoặc kết hợp với vector search. Khi kết hợp, full text search giúp lọc bớt kết quả không liên quan dựa trên từ khóa cụ thể, còn vector search tìm các đoạn có nghĩa gần với câu hỏi. Trong phiên bản hiện tại, project

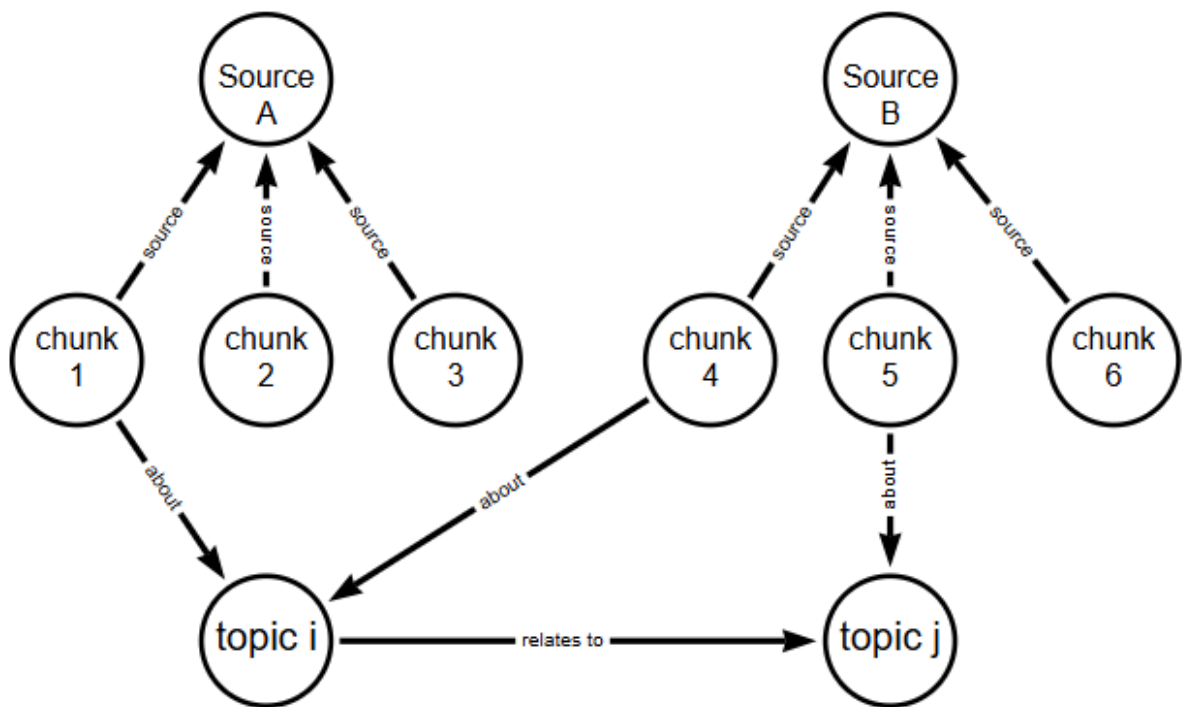
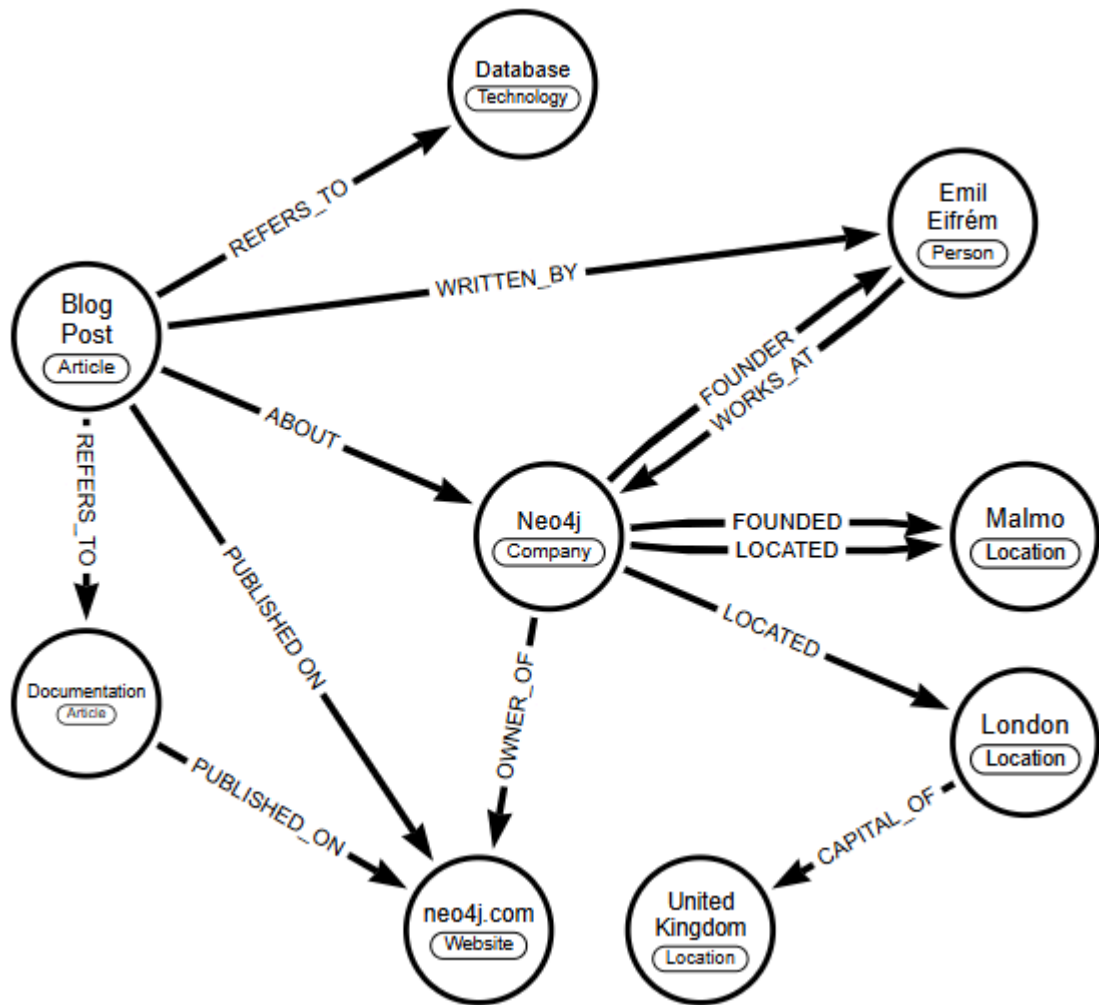
tập trung vào vector index và Cypher; trong hướng phát triển, có thể bổ sung full-text index cho các thuộc tính id, name và text để tăng khả năng tìm chính xác tên thực thể lịch sử.

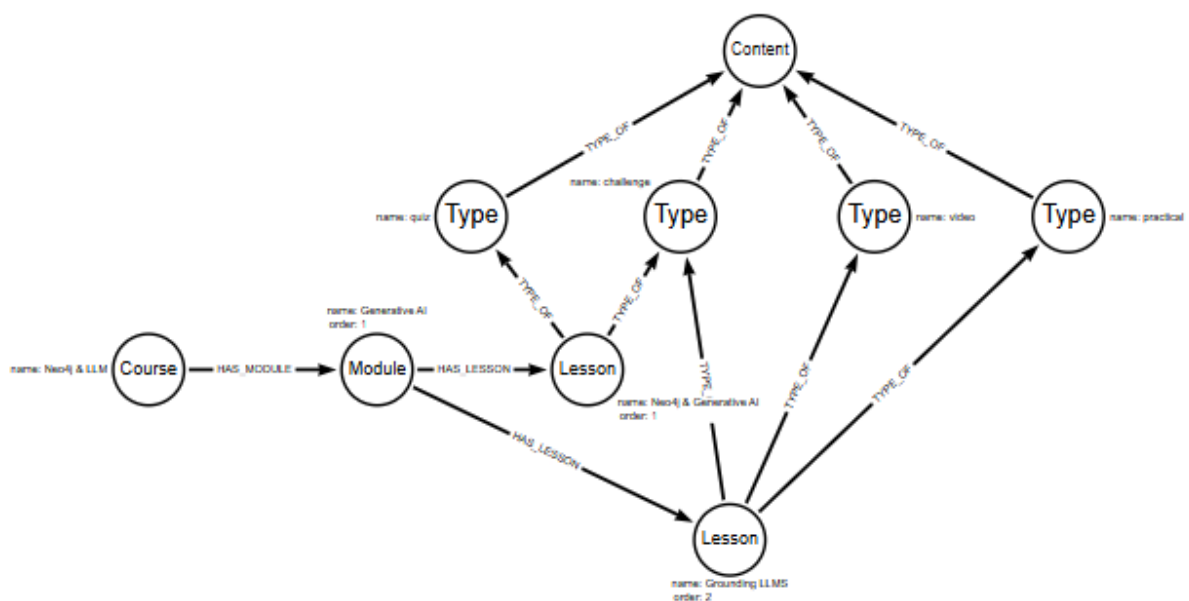
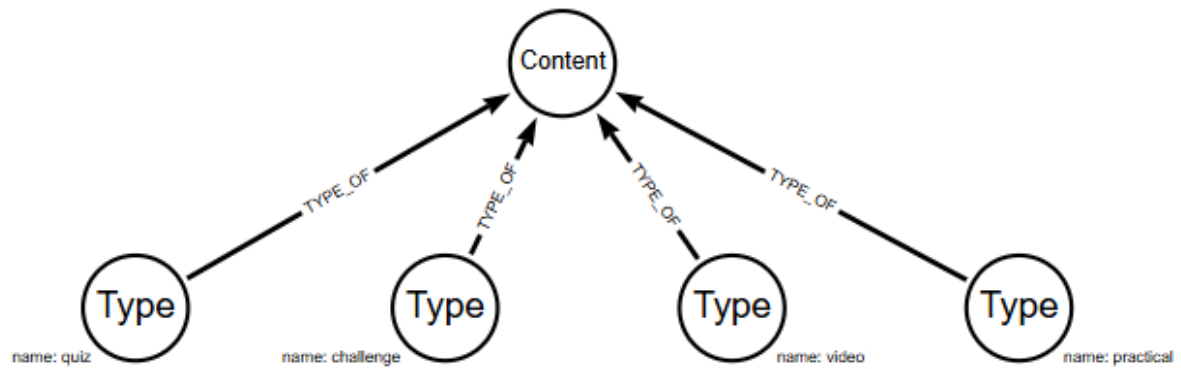
## **2.12. Knowledge Graph**

Knowledge Graph là phương pháp biểu diễn tri thức dưới dạng các thực thể, thuộc tính và mối quan hệ. Một graph thường gồm node, relationship và property. Node biểu diễn thực thể như nhân vật, địa danh, tổ chức, sự kiện; relationship biểu diễn quan hệ có hướng giữa các thực thể; property lưu thông tin mô tả như tên, loại thực thể hoặc nội dung văn bản.

Knowledge Graph hữu ích cho ứng dụng Generative AI vì nó cung cấp dữ liệu có cấu trúc, liên kết và dễ truy vấn. Nhờ có đồ thị, hệ thống không chỉ biết một đoạn văn nói gì, mà còn biết các thực thể trong đoạn văn có quan hệ như thế nào. Điều này giúp nâng cao ngữ cảnh, khả năng suy luận và độ chính xác của câu trả lời.

Một đặc điểm quan trọng của Knowledge Graph là khả năng tích hợp dữ liệu từ nhiều nguồn khác nhau. Dữ liệu từ tài liệu, API, cơ sở dữ liệu hoặc file văn bản có thể được chuyển thành node và relationship trong cùng một đồ thị. Khi dữ liệu tăng lên, graph có thể mở rộng bằng cách thêm node, edge, loại thực thể hoặc nguyên tắc tổ chức mới.





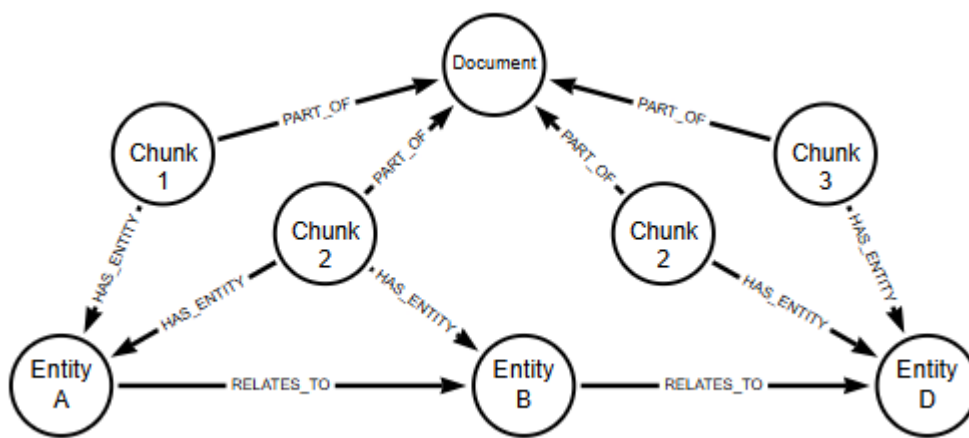
Trong project, knowledge graph được thiết kế đơn giản nhưng phù hợp với phạm vi đề án. Các thực thể lịch sử được lưu bằng nhãn Entity với thuộc tính id, name và type. Các mối quan hệ được lưu bằng relationship type trong Neo4j, ví dụ một thực thể “Mỹ” có thể có quan hệ “âm mưu thay thế” với “Pháp” hoặc “xâm chiếm” với “miền Nam”.

### 2.13. Tạo Knowledge Graph từ dữ liệu phi cấu trúc

Dữ liệu phi cấu trúc như văn bản, trang web hoặc PDF là nguồn dữ liệu phong phú để xây dựng knowledge graph. Tuy nhiên, việc tạo đồ thị từ dữ liệu phi cấu trúc thường phức tạp vì cần nhiều bước: thu thập, làm sạch, tách đoạn, phân tích, trích xuất thực thể, trích xuất quan hệ và chuyển đổi sang mô hình đồ thị.

Quy trình phổ biến để xây dựng knowledge graph từ văn bản phi cấu trúc bằng LLM như sau:

1. Thu thập dữ liệu từ nhiều nguồn và nhiều định dạng khác nhau.
2. Tách dữ liệu thành các chunk nhỏ để LLM có thể xử lý hiệu quả.
3. Vector hóa dữ liệu bằng embedding nếu hệ thống cần tìm kiếm ngữ nghĩa.
4. Đưa từng chunk vào LLM để trích xuất thực thể và quan hệ theo ràng buộc đã định nghĩa.
5. Tạo graph bằng cách chuyển đầu ra của LLM thành các node và relationship trong cơ sở dữ liệu đồ thị.



## 2.14. Neo4j và Cypher

Neo4j là hệ quản trị cơ sở dữ liệu đồ thị, lưu dữ liệu theo mô hình node - relationship - property. Neo4j phù hợp với các bài toán cần biểu diễn và truy vấn quan hệ phức tạp. Với dữ liệu lịch sử, Neo4j giúp mô hình hóa mối quan hệ giữa nhân vật, tổ chức, địa danh, sự kiện và văn kiện lịch sử một cách trực quan hơn so với bảng quan hệ truyền thống.

Cypher là ngôn ngữ truy vấn của Neo4j. Cypher cho phép mô tả mẫu quan hệ trong đồ thị bằng cú pháp gần với hình vẽ node và edge. Ví dụ, một truy vấn có thể tìm tất cả các quan hệ đi ra từ node “Mỹ” đến các thực thể liên quan. Trong project, LLM được hướng dẫn sinh Cypher theo schema của graph để truy xuất dữ liệu phù hợp với câu hỏi người dùng.

Mô hình dữ liệu cơ bản của project:

```
(:Entity {id, name, type}) -[:RELATION_TYPE]-> (:Entity {id, name, type})
(:Chunk {id, text, embedding})
VECTOR INDEX chunk_vector FOR (c:Chunk) ON (c.embedding)
```

## 2.15. Streamlit, LangChain và các công nghệ sử dụng

Streamlit là thư viện Python dùng để xây dựng giao diện web nhanh, phù hợp cho demo, đồ án và các ứng dụng dữ liệu. Trong project, Streamlit tạo giao diện dạng chat, hiển thị lịch sử hội thoại, câu trả lời của chatbot và phần nhật ký truy vấn để kiểm tra dữ liệu graph/vector đã sử dụng.

LangChain là framework hỗ trợ xây dựng ứng dụng dùng LLM. Project sử dụng LangChain để kết nối với OpenAI, Neo4jGraph, Neo4jVector và GraphCypherQChain. Nhờ LangChain, các bước gọi LLM, tạo prompt, truy vấn vector và truy vấn graph được tổ chức theo pipeline rõ ràng hơn.

OpenAI API được dùng cho hai nhiệm vụ: sinh embedding bằng text-embedding-3-small và sinh câu trả lời bằng GPT-4o-mini. Ngoài ra, project dùng Open AI cho hướng trích xuất graph.

| Nhóm công nghệ       | Công nghệ / thư viện                         | Vai trò trong hệ thống                                                       |
|----------------------|----------------------------------------------|------------------------------------------------------------------------------|
| Ngôn ngữ lập trình   | Python                                       | Xây dựng pipeline xử lý dữ liệu, import database và ứng dụng chatbot.        |
| Giao diện            | Streamlit                                    | Tạo web app hỏi đáp dạng chat, hiển thị câu trả lời và nhật ký truy vấn.     |
| LLM                  | GPT-4o-mini                                  | Sinh câu trả lời cuối cùng và hỗ trợ text-to-Cypher trong GraphCypherQChain. |
| Embedding            | text-embedding-3-small                       | Biến các chunk văn bản thành vector 1536 chiều phục vụ semantic search.      |
| Cơ sở dữ liệu đồ thị | Neo4j                                        | Lưu Entity, relationship, Chunk và vector index trong cùng một hệ quản trị.  |
| Truy vấn đồ thị      | Cypher                                       | Truy vấn quan hệ giữa nhân vật, tổ chức, sự kiện, địa danh.                  |
| Framework LLM        | LangChain, langchain-neo4j, langchain-openai | Tích hợp LLM, graph, vector store và prompt chain.                           |
| Tiền xử lý văn bản   | RecursiveCharacterTextSplitter               | Tách tài liệu thành chunk có chồng lấn để giữ ngữ cảnh.                      |
| Quản lý cấu hình     | python-dotenv, file .env                     | Lưu API key và thông tin kết nối Neo4j ngoài mã nguồn.                       |

|                      |        |                                                                                           |
|----------------------|--------|-------------------------------------------------------------------------------------------|
| Định dạng trung gian | JSON   | Lưu processed_chunks.json và extracted_graph_data.json để tách pipeline thành nhiều bước. |
| LLM trích xuất graph | OpenAI | Đọc chunk và trích xuất thực thể, quan hệ dưới dạng node/edge.                            |

## 2.16. Ý nghĩa của cơ sở lý thuyết đối với project

Các lý thuyết trên giải thích vì sao project không xây dựng chatbot theo cách hỏi trực tiếp LLM thông thường. Nếu chỉ hỏi LLM, hệ thống có nguy cơ trả lời sai hoặc không bám sát tài liệu lịch sử đã chọn. Bằng cách dùng RAG, hệ thống truy hồi các chunk có liên quan. Bằng cách dùng Knowledge Graph và GraphRAG, hệ thống khai thác thêm quan hệ giữa các thực thể lịch sử. Bằng cách dùng Neo4j vector index, hệ thống có thể tìm kiếm ngữ nghĩa trên dữ liệu tiếng Việt. Bằng cách dùng prompt ràng buộc, hệ thống yêu cầu mô hình trả lời dựa trên dữ liệu truy xuất và hạn chế hallucination.

Như vậy, phân cơ sở lý thuyết và công nghệ sử dụng cho thấy kiến trúc Hybrid GraphRAG của project là phù hợp với bài toán chatbot học lịch sử Việt Nam. Kiến trúc này tận dụng ưu điểm điển đạt tự nhiên của LLM, khả năng tìm kiếm ngữ nghĩa của embedding, khả năng biểu diễn quan hệ của Knowledge Graph và khả năng triển khai linh hoạt của Neo4j, LangChain, Streamlit.

# CHƯƠNG 4. KẾT QUẢ CÀI ĐẶT VÀ THỬ NGHIỆM

## 4.1. Môi trường triển khai

Hệ thống được phát triển và thử nghiệm trên môi trường sau:

| Thành phần      | Thông số                           |
|-----------------|------------------------------------|
| Hệ điều hành    | Windows 11                         |
| Python          | 3.11                               |
| Neo4j Aura      | Free Tier — Cloud (GCP)            |
| LLM chính       | GPT-4o-mini (OpenAI API)           |
| Embedding Model | text-embedding-3-small (1536 dims) |
| Framework UI    | Streamlit 1.35+                    |
| LangChain       | v0.2.x + langchain_neo4j           |

## 4.2. Kết quả xây dựng kho dữ liệu

### 4.2.1. Thống kê dữ liệu thô

- Tổng số tài liệu đầu vào: 6 file.
- Tổng số chunks sau khi cắt: 251 đoạn văn (chunk\_size=800, overlap=150).
- Tổng số nodes :Entity trong đồ thị: ~1.200 – 1.500 nodes.
- Tổng số relationships: 2841 cạnh với hàng chục loại quan hệ khác nhau.
- Tổng số nodes :Chunk (Vector Store): 251 nodes với embedding 1536 chiều mỗi node.

## Free instance

● RUNNING

ID: f863f2ed 

Type: AuraDB Free   Nodes: 2,181 (1%)   Relationships: 2,841 (1%)

### 4.2.2. Chất lượng đồ thị tri thức

Đồ thị tri thức được xây dựng tự động thông qua AI, nên chất lượng phụ thuộc vào khả năng trích xuất của mô hình. Kiểm tra thủ công trên Neo4j Browser cho thấy các thực thể chính được nhận diện chính xác: Hồ Chí Minh, Ngô Đình Diệm, Chiến dịch Hồ Chí Minh, Hiệp định Paris, Sài Gòn, Bến Tre, Mặt trận Dân tộc Giải phóng miền Nam Việt Nam. Các quan hệ logic như CÓ\_VAI\_TRO\_QUYẾT ĐỊNH\_TRỰC\_TIẾP, CÓ\_VAI\_TRÒ\_QUYẾT ĐỊNH được gán đúng hướng và đúng ngữ cảnh cho phần lớn trường hợp.

